

Computer Applications in Engineering Design (CAED) EC-103

Dr. Aimal Khan

Ph.D. Communications Engineering,
University of Rostock, Germany

Master in Digital Communications
University of Kiel, Germany

BE Computer Engineering, College of EME,
NUST

Assistant Professor: Department of
Computer Engineering, College of EME,
NUST



Dr. Aimal Khan



In 1946 the first computer was announced. It was called ENIAC, which stood for Electronic Numerical Integrator and Computer. ENIAC took up a large room and required several people to operate. She operated with vacuum tubes and could output 5,000 addition problems in one second. The computer used 170,000 watts of power, weighed 28 tons, had 17,840 vacuum tubes, filled a large building and cost \$487,000. (By today's standards, that is about ten million dollars!) It took several people to run her. It was basically a larger super fast calculator with some input features via punch cards. You think you have computer problems today? With 17,480 tubes, ENIAC required constant maintain as there was always a tube going out. On average, one would go out about every other day. It took about fifteen minutes to locate the bad tube. It's longest continuous period of operation without a failure was 116 hours. Aside from the 17,480 vacuum tubes, ENIAC had 7,200 crystal diodes, 1,500 relays, 70,000 resistors, 10,000 capacitors and around 5 million hand-soldered joints. The typical computer you use today is of course tens of thousands of times faster.

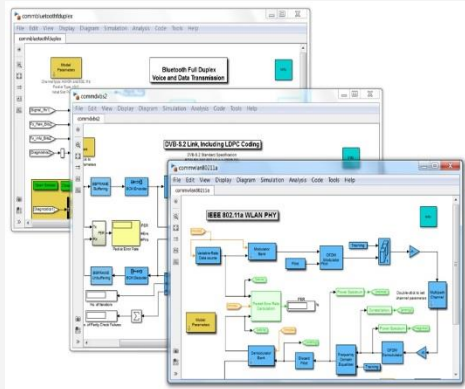
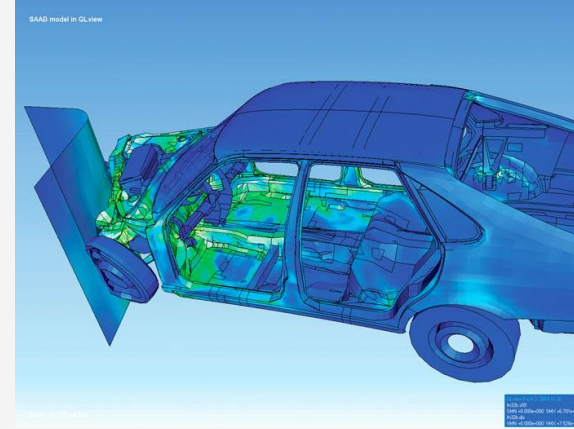
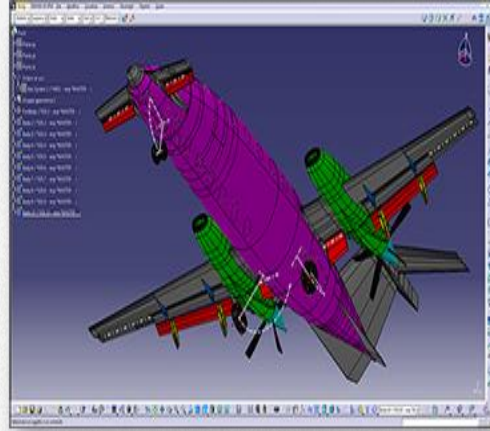
Modern PC



What this course is about

- To learn how to use a computer to perform an engineering and scientific job, research, or project efficiently
- The focus is on the different softwares as tools used

Some of the applications in Engineering



Course Schedule

- 14-16 lectures
- 4-6 Quizzes
- 2-4 home assignments
- 2 Sessional Exams
- Practical Work
- Final Exam

Score/Grading

- Tentative marks division
 - 2 Sessional Exams 25~30%
 - Assignments 5~10%
 - Quizzes 5~10%
 - Practical 33.33%
 - Lab
 - Final Examination 40~45%

Final Objectives

- To introduce engineering drawing concepts using various tools required for software & hardware designing
- To provide a guideline for making of software designs using Visio and Rational Rose for understanding and implementing object oriented designs and standards like UML.
- How to use Matlab for plotting, analysis, and simulations

Course Outline

- Be able to implement engineering drawing concepts using various tools required for software & hardware desing.
- Have thorough knowledge of software like Visio for software drawing like process diagrams, class diagrams, sequence diagrams, interaction diagrams, deployment diagrams and entity relationship diagram
- Have thorough knowledge of software like Matlab for plotting, data and mathematical analysis, simulation.
- Will have an understanding of computer aided drafting and scientific writing principles and practices and the respective software like Latex, Lyx, etc.
- Will be able to understand how theoretical concepts of electronic components and circuits can be implemented using simulation software like Matlab and Labview.

Syllabus

- Week 1
 - Introduction to MATLAB
 - GUI
 - Variables, operations, built in functions
 - Help
- Week 2
 - Programming
 - Syntax
 - Arrays: Numeric, cell, structure
 - Strings
 - Built in functions (i.e. find, mean, max, min, sum, etc)
 - File I/O (text, binary, netCDF, HDF)
 - Basic program organization
 - Debugging

Chapter 1: MATLAB

Chapter 1 Matlab

1.1 Introduction into Matlab and Main Features

1.2 Windows in Matlab

1.3 Commands in Matlab

1.4 Operations in Matlab

1.1 What is MATLAB?

- MATLAB stands for MATrix LABoratory.
- MATLAB is a simple programming language with its own extensive library of mathematical and graphical subroutines
- It integrates computation and graphics in one easy to use interface
- MATLAB is very extendable. There are many add-ons (toolboxes) for specific requirements

1.1 What is MATLAB?

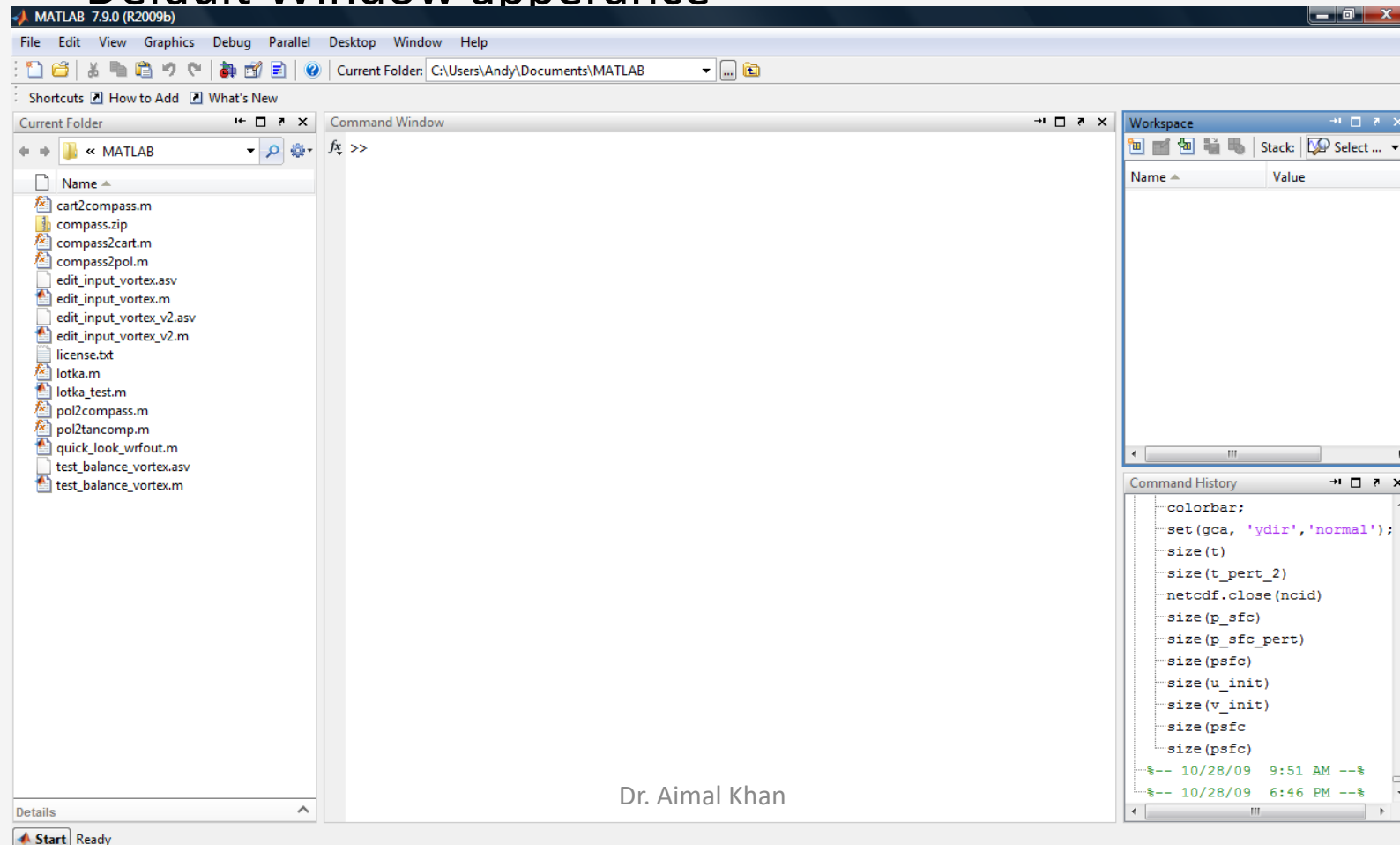
- Main Features
 - Simple programming rules
 - integer, real and complex values
 - Comprehensive mathematical library
 - Extensive graphics tools
 - Linkages with other languages

1.1 What is MATLAB?

- If MATLAB can process and visualize data, why ever use FORTRAN, C, or some other language?
- MATLAB is an interpreted language
 - It is not a compiled language
 - Therefore identical code executes more slowly, sometimes MUCH more slowly in MATLAB
 - MATLAB has more memory overhead than equivalent FORTRAN or C programs

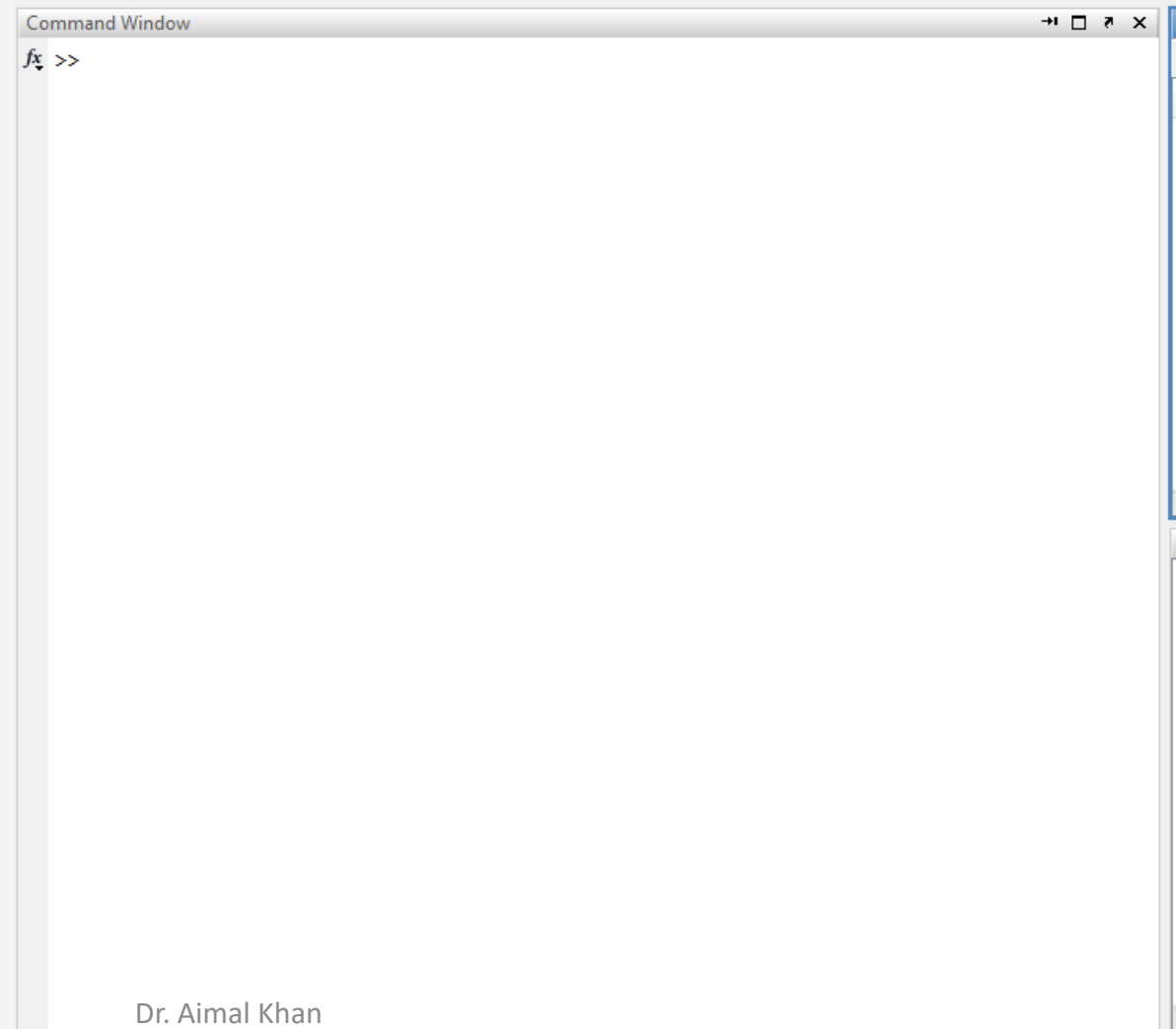
1.2 Starting MATLAB

- Once MATLAB is running the GUI (Graphical User Interface) will appear
 - Default Window apperance



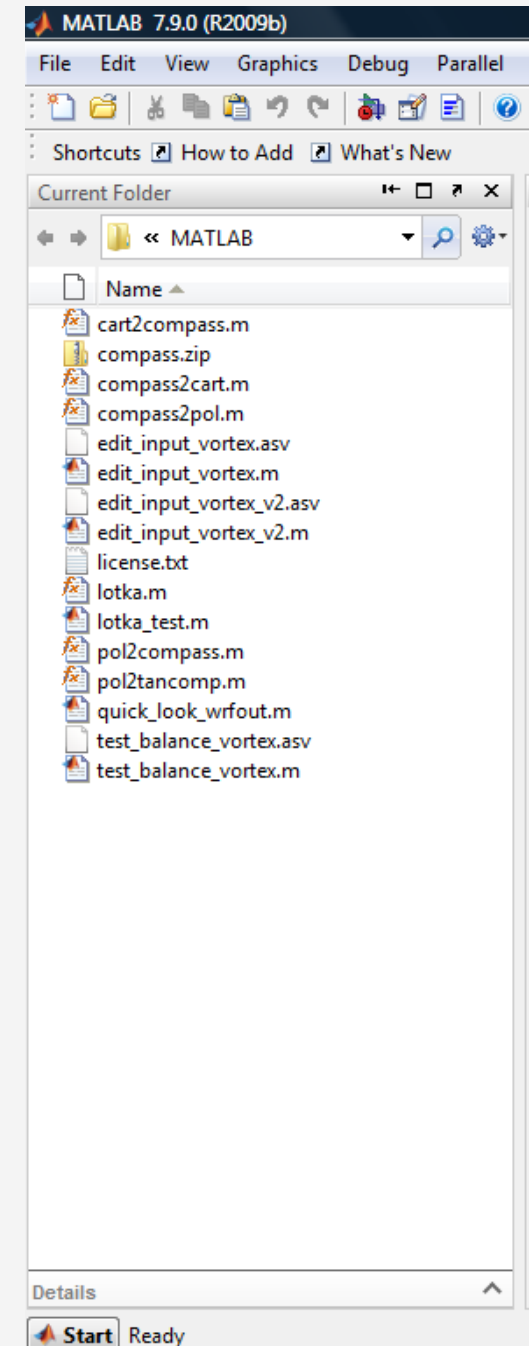
1.2 Starting MATLAB

- Command Window
 - Main window in MATLAB
 - Commands entered here



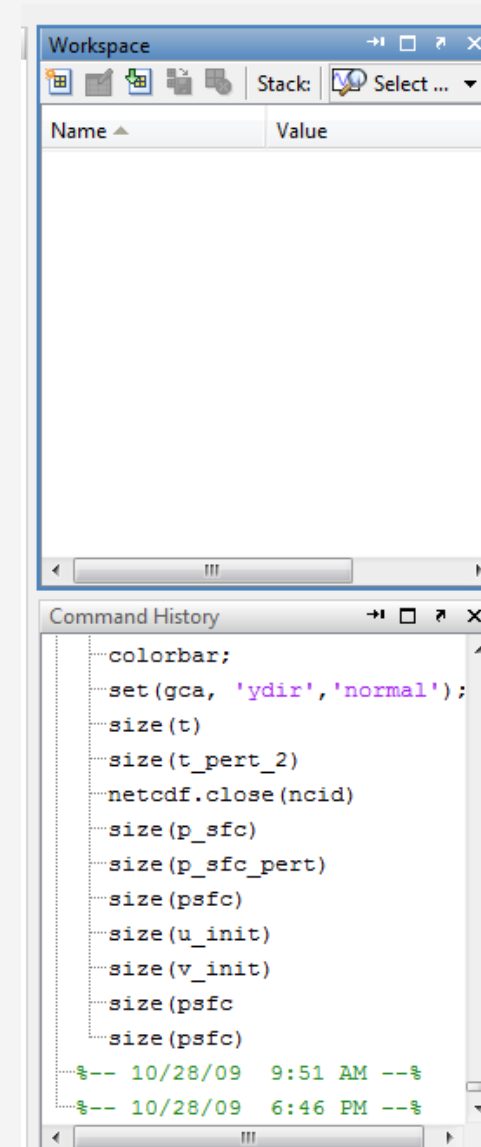
1.2 MATLAB GUI

- Current Folder Window
 - Displays contents of the current working directory
- MATLAB Search Path
 - Path that MATLAB uses to search for script and function files
 - Default path contains all built in MATLAB functions

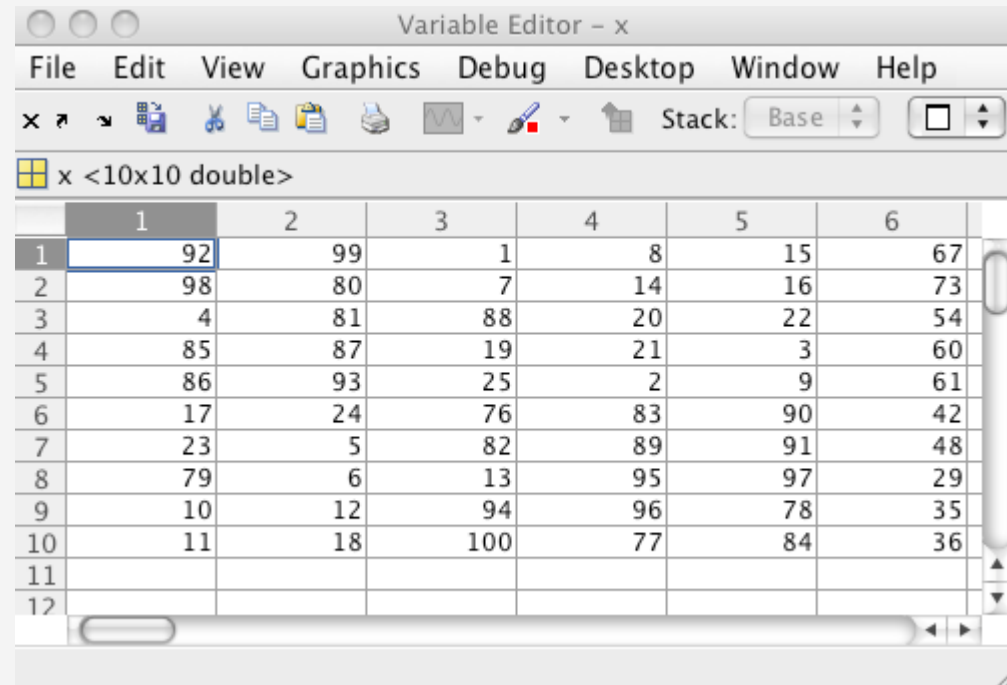


1.2 MATLAB GUI

- Workspace Window
 - Shows all currently defined variables
 - Array dimensions
 - Min, max values
 - Good debugging tool
- Command History
 - Shows all past commands
 - Can copy and past commands into command window
 - Double click will execute command



1.2 Variable editor



The image shows a 'Variable Editor' window for a variable 'x' of type '<10x10 double>'. The window has a menu bar (File, Edit, View, Graphics, Debug, Desktop, Window, Help) and a toolbar with various icons. The main area displays a table with 10 rows and 10 columns. The first row is highlighted, and the cell at row 1, column 1 (value 92) is selected. The table contains numerical data for the first 10 rows and columns.

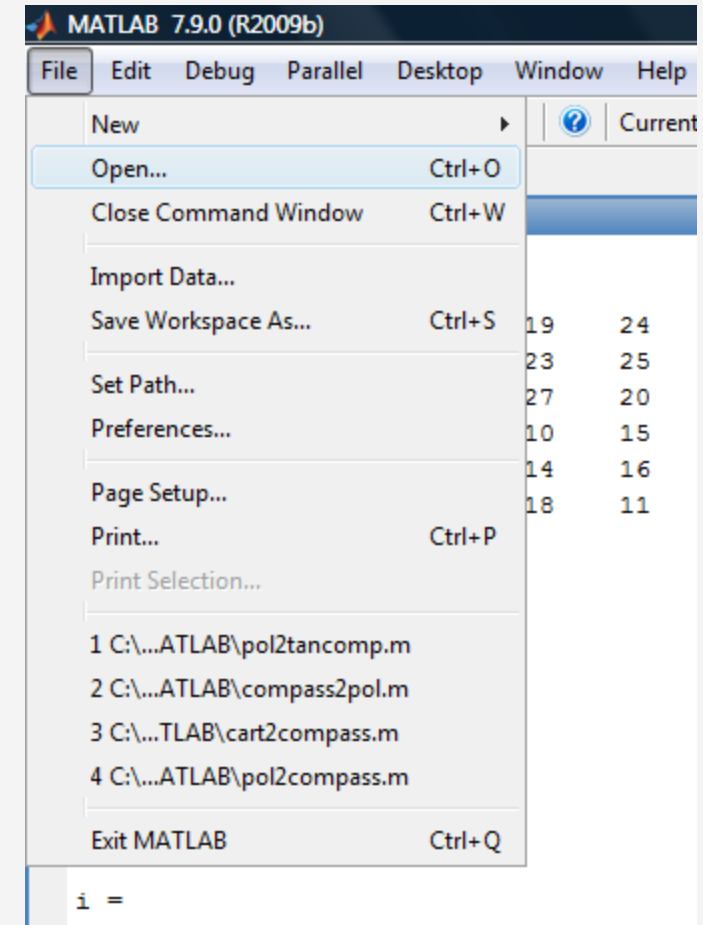
	1	2	3	4	5	6	7	8	9	10
1	92	99	1	8	15	67				
2	98	80	7	14	16	73				
3	4	81	88	20	22	54				
4	85	87	19	21	3	60				
5	86	93	25	2	9	61				
6	17	24	76	83	90	42				
7	23	5	82	89	91	48				
8	79	6	13	95	97	29				
9	10	12	94	96	78	35				
10	11	18	100	77	84	36				
11										
12										

1.2 MATLAB GUI

- Other windows include editor window, figure window, help, etc

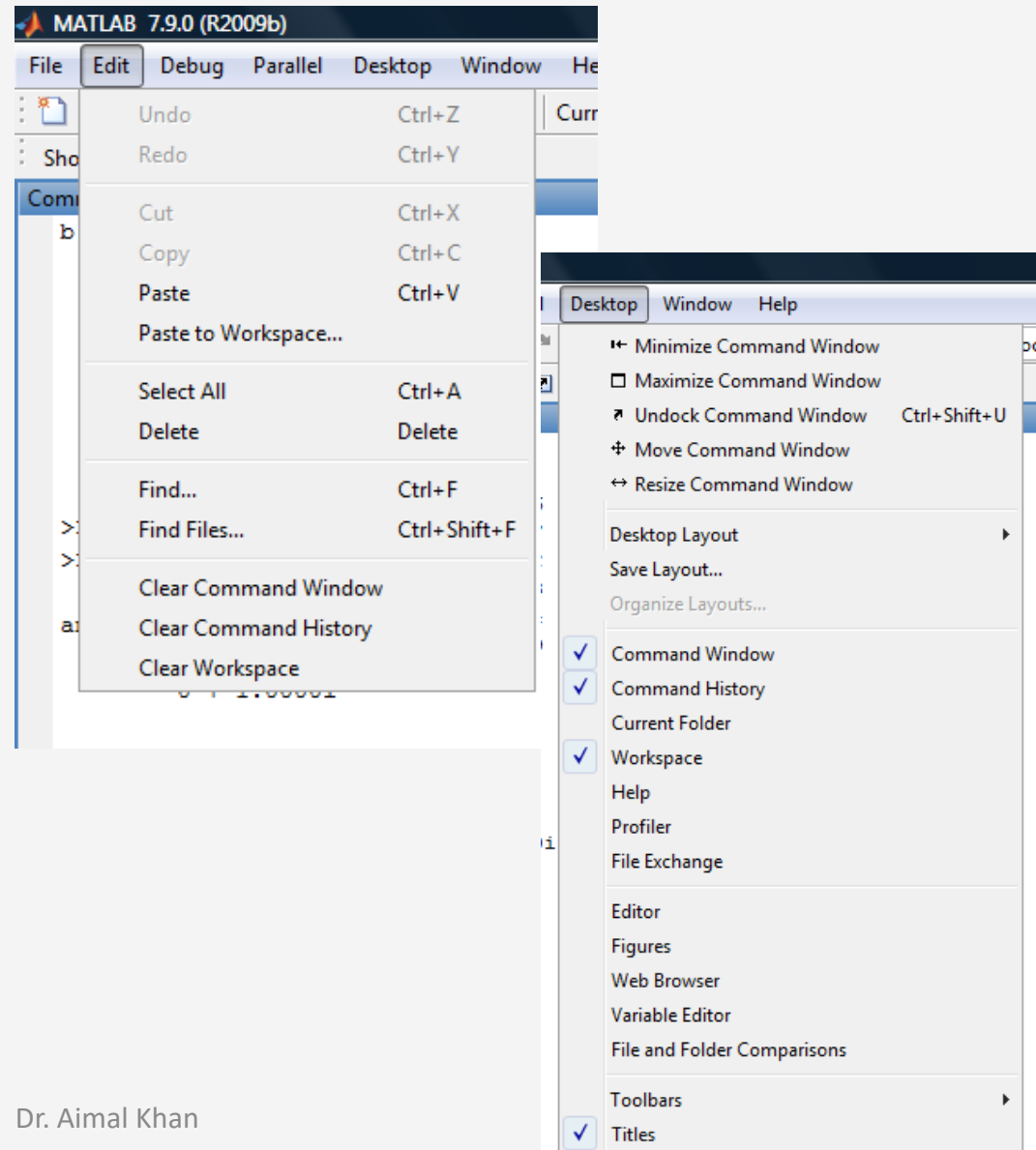
1.2 MATLAB GUI

- Desktop Menus
- File Menu
 - New
 - Create new MATLAB program file (m-file)
 - Open existing m-file
 - Open recent m-files



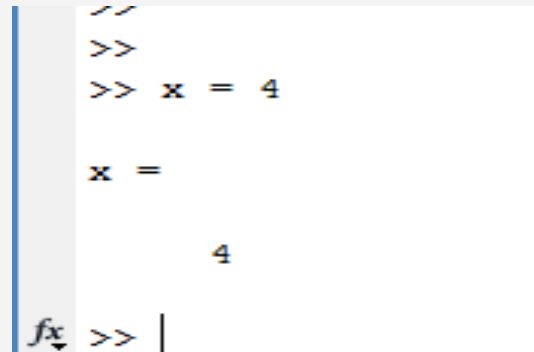
1.2 MATLAB GUI

- Edit Menu
 - Copy, cut, paste
 - Find and replace phrases
 - Clear command history, workspace
- Desktop Menu
 - Change appearance of desktop
 - Select windows to display



1.3 Interactive Commands

- Enter commands at >> prompt

A screenshot of the MATLAB command window. It shows a sequence of commands and their output. The first line is a comment '//'. The second line is the prompt '>>'. The third line is the command '>> x = 4'. The fourth line shows the output 'x =' followed by a space. The fifth line shows the output '4'. The sixth line shows the prompt 'fx>>' followed by a vertical cursor bar.

```
//>>>> x = 4x = 4fx>> |
```

- Variable 'x' automatically allocated
 - MATLAB does not require declaration of variables
 - Nice, but can get you in trouble so be careful

1.3 Interactive Commands

- MATLAB is case sensitive
 - $X = 4$, $x = 1$ are two different vars.
 - Variable 'ans' will take value of result of command if no equal sign specified
 - Holds most recent result only
 - Semicolon at end of line will suppress output, it is not required like in C
 - Useful in script files and debugging

```
//  
>> b = 3  
  
b =  
  
3  
  
>> c = 2  
  
c =  
  
2  
  
>> b - c  
  
ans =  
  
1  
  
>> b - c;
```

1.3 Interactive Commands

- Format of output
 - Defaults to 4 decimal places
 - Can change using **format** statement
 - **format long** changes output to 15 decimal places

```
>> pi

ans =

    3.1416

>> format long
>> pi

ans =

    3.141592653589793
```

1.4 Operators

- Scalar arithmetic operations

Operation	MATLAB form	
• Exponentiation: a^b	a^b	<code>a^b</code>
• Multiplication: $a * b$	$a * b$	<code>a*b</code>
• Division: $a / b = a/b$	a/b	<code>a/b</code>
• Addition: $a + b$	$a + b$	<code>a+b</code>
• Subtraction: $a - b$	$a - b$	<code>a-b</code>

- MATLAB will ignore white space between variables and operators

1.4 Order of Operations

- Parentheses
- Exponentiation
- Multiplication and division have equal precedence
- Addition and subtraction have equal precedence
- Evaluation occurs from left to right
- When in doubt, use parentheses
 - MATLAB will help match parentheses for you

1.4 Logical Operators for Scalars

Operator	Example
>	<code>r > 0</code>
<	<code>x < 1</code>
>=	<code>r >= x</code>
<=	<code>x <= eps</code>
==	<code>p == pi</code>
~=	<code>p ~= pi</code>
& (and)	<code>a >3 & c<0</code>
(or)	<code>g >3 g<0</code>

Chapter 2

2.1 Variables in Matlab

2.2 Arrays in Matlab

2.3 Accessing Elements of Arrays

2.4 Numerical Arrays Operations

2.5 Strings

2.1 Variables

- MATLAB is case sensitive
 - X is not equal to x
- Variable names must start with a letter
 - You'll get an error if this doesn't happen
 - After that, can be any combination of letters, numbers and underscores
 - No special characters though

```
//  
>> x = 4  
  
x =  
  
4  
  
>> X = 7  
  
X =  
  
7  
  
>> x-X  
  
ans =  
  
-3
```


2.1 Variables

- Variable types
 - Numeric
 - Logical
 - Character and string
 - Function handle

2.1 Variables

- Don't name your variables the same as Matlab keywords
 - for, while, if, case, switch, etc
 - Use `iskeyword('str')` to know if str is keyword
- Don't name your variables the same as functions
 - min, max, sqrt, cos, sin, tan, mean, median, etc
- Avoid using MATLAB reserved words
 - i, j, eps, nargin, end, pi, date, etc
 - i, j : can be used as counter variables but it degrades the performance so better avoid them

2.1 Complex Scaler Variables

- Use i or j to represent the imaginary part
- To write a complex number c with real part equal to 3 and imaginary part equal to 5, write

$$c = 3+5j$$

- You can also use complex function e.g. `Complex(3,5)`
- Use function `real` to get real part, e.g. `real(c) = 3`
- Use function `imag` to get imaginary part e.g. `Imag(c) = 5`
- Use function `isreal` to know if a number is real or complex
- Use `conj` function to take conjugate of a number

2.1 Numeric Data Types in Matlab

- Format of output
 - By default double-precision floating-point numbers (64 bits)
 - However you can change the data type by the below functions.

[double](#)

Convert to double precision (64)

[single](#)

Convert to single precision (32)

[int8](#)

Convert to 8-bit signed integer

[int16](#)

Convert to 16-bit signed integer

[int32](#)

Convert to 32-bit signed integer

[int64](#)

Convert to 64-bit signed integer

[uint8](#)

Convert to 8-bit unsigned integer

[uint16](#)

Convert to 16-bit unsigned integer

[uint32](#)

Convert to 32-bit unsigned integer

[uint64](#)

Convert to 64-bit unsigned integer

MATLAB Help

- Ways to get help in MATLAB
 - help **function name**
 - Provides basic info about the function
- Type **helpwin** on command line
- Look under the help menu on the desktop

```
>> help sqrt|
SQRT    Square root.
        SQRT(X) is the square root of the elements of X. Complex
        results are produced if X is not positive.

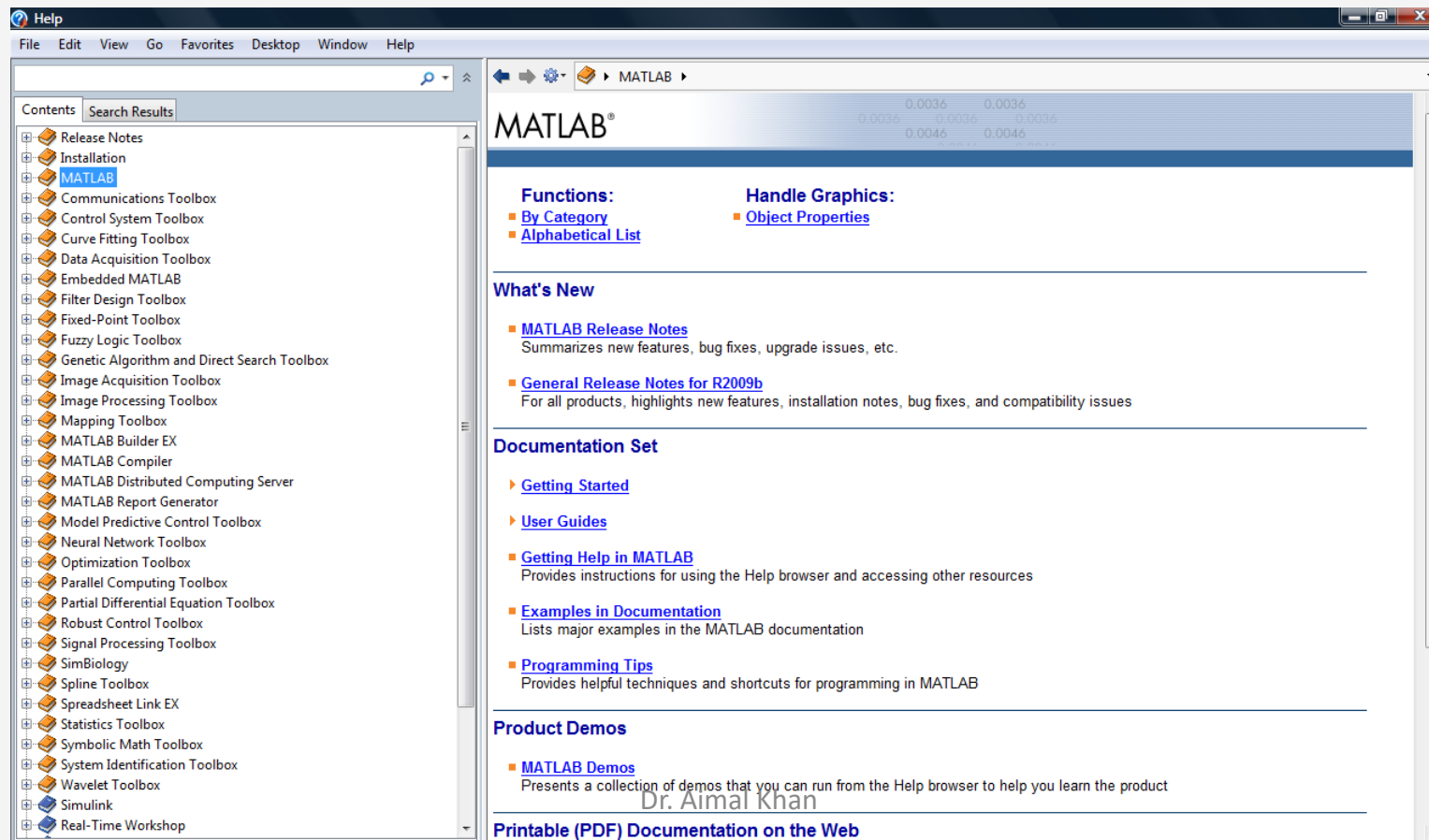
        See also sqrtm, realsqrt, hypot.

        Overloaded methods:
            codistributed/sqrt

        Reference page in Help browser
            doc sqrt
```

MATLAB Help

- Product help window
 - Help>product help



2.2 Arrays

- MATLAB is adept at handling arrays
 - Optimized for vector/matrix operations
 - This allows for a reduction of code in some cases
- Array types: Numeric, character, logical, function handle
 - Numeric types: single, double, int8, int16, int32, uint8, uint16, uint32

2.2 Numeric Arrays

- One dimensional arrays, called vectors
 - Can create row or column vectors
 - Row vector
 - A few ways to create a row vector

```
Command window
>> foo = [3 4 5 6]

foo =

     3     4     5     6
```

```
>> foo = [3,4,5,6]

foo =

     3     4     5     6
```


2.2 Numeric Arrays

- Column vectors
 - A few ways to generate column vectors too
 - Semicolon between elements starts new row
 - Transpose a row vector, or use `return` between elements

```
>> bar = [3;4;5;6]
```

```
bar =
```

```
3  
4  
5  
6
```

```
fx \
```

```
>> bar = [3,4,5,6]'
```

```
bar =
```

```
3  
4  
5  
6
```

Dr. Aimal Khan

```
>> bar = [3
```

```
4
```

```
5
```

```
6]
```

```
bar =
```

```
3  
4  
5  
6
```

2.2 Numeric Arrays

- Two-dimensional arrays, called a matrix in MATLAB often
- Size = rows by columns
 - `[r c] = size(array_name)` if `array_name` is a matrix
 - **size** will work for n-dimension arrays and output size of each dimension into a row vector
- Basic matrix creation:

```
>> foo = [3,4,5;6,7,8]
```

```
foo =
```

3	4	5
6	7	8

2.2 Numeric Arrays

- Array addressing
 - Vector: foo
 - `foo(:)` gives all row or column elements
 - `foo(1:5)` gives the first five row or column elements
 - `foo(2)` gives the second element
 - Matrix: bar
 - `bar(1,3)` gives the first row, third column element
 - `Bar(:,2)` gives all elements in the second column
 - `Bar(1,:)` gives all elements in the first row
 - `Bar(3:4,1:3)` gives all elements in the third and fourth rows that are in the first through third columns

Example

```
v = [16 5 9 4 2 11 7 14];
```

The subscript can be a single value:

```
v(3)      % Extract the third element  
ans =  
     9
```

Or the subscript can itself be another vector:

```
v([1 5 6]) % Extract the first, fifth, and sixth elements  
ans =  
    16     2    11
```

The colon notation in MATLAB provides an easy way to extract a range of elements from v:

```
v(3:7)     % Extract the third through the seventh elements  
ans =  
     9     4     2    11     7
```

Example

```
v(end)    % Extract the last element
ans =
    14
```

The `end` operator can be used in a range:

```
v(5:end)    % Extract the fifth through the last elements
ans =
     2    11     7    14
```

You can even do arithmetic using `end`:

```
v(2:end-1)    % Extract the second through the next-to-last elements
ans =
     5     9     4     2    11     7
```

Combine the colon operator and `end` to achieve a variety of effects, such as extracting every k-th element or flipping the entire vector:

```
v(1:2:end)    % Extract all the odd elements
ans =
    16     9     2     7
v(end:-1:1)    % Reverse the order of elements
ans =
    14     7    11     2     4     9     5    16
```

By using an indexing expression on the left side of the equal sign, you can *replace* certain elements of the vector:

```
v([2 3 4]) = [10 15 20]    % Replace some elements of v
v =
    16    10    15    20     2    11     7    14
```

Example

```
A =  
    16     2     3    13  
     5    11    10     8  
     9     7     6    12  
     4    14    15     1
```

Most often, indexing in matrices is done using two subscripts—one for the rows and one for the columns. The simplest form just picks out a single element:

```
A(2,4) % Extract the element in row 2, column 4  
ans =  
     8
```

More generally, one or both of the row and column subscripts can be vectors:

```
A(2:4,1:2)  
ans =  
     5    11  
     9     7  
     4    14
```

Example

- What $A(5)$ is ?
- In Two dimensional array counting goes column-wise
- So here $A(5) = 2$

2.4 Numeric Arrays Operations

- Array operations
 - MATLAB has array or element by element and matrix operations when dealing with arrays
- Element-by-element operations
 - Multiplying a vector or array by a scalar is an easy example

```
foo =  
  
     3     4     5  
     6     7     8  
  
>> foo*2  
  
ans =  
  
     6     8    10  
    12    14    16
```


2.4 Numeric Arrays

- How do you multiply to arrays element-by-element?
 - Do it in a for loop
 - Very slow, avoid whenever possible
 - Use element-by-element operations
 - Addition and subtraction are the same: +, -
 - Multiplication, right and left division and exponentiation use “.” before the symbol
 - i.e. `foo.*bar` or `foo./bar` or `foo.^3`

2.4 Numeric Arrays Operations

- Examples of element-by-element operation results

```
>> foo = [1 2; 1 2];  
>> bar = [3 4; 3 4];  
>> foo.*bar
```

```
ans =
```

```
    3    8  
    3    8
```

```
>> foo./bar
```

```
ans =
```

```
    0.3333    0.5000  
    0.3333    0.5000
```

```
>> foo+bar
```

```
ans =
```

```
    4    6  
    4    6
```

```
>> foo.^2
```

```
ans =
```

```
    1    4  
    1    4
```

2.4 Numeric Arrays Operations

- Matrix operations
 - Matrix operations are the same symbols as standard scalar operations
 - Apply when multiplying or dividing matrices

```
>> col = [1 2 3 4]'  
  
col =  
  
     1  
     2  
     3  
     4  
  
>> row = [1 2 3 4]  
  
row =  
  
     1     2     3     4  
  
>> row*col  
  
ans =  
  
    30
```

```
>> col*row  
  
ans =  
  
     1     2     3     4  
     2     4     6     8  
     3     6     9    12  
     4     8    12    16
```

```
>> foo*bar  
  
ans =  
  
     9    12  
     9    12
```

2.4 Numeric Arrays Operations

- Useful built-in functions for working with arrays
 - **Num_of_elements = length(array)**
 - Returns the length of the longest dimension
 - **Max_val = max(array)**
 - Returns the largest element in the array if it is a vector
 - Returns a row vector of the largest element if the array is a matrix
 - **min_val = min(array)**
 - Same as **max**, but returns the minimum values

2.4 Numeric Arrays Operations

- **sum(array)**
 - Returns the sum of a vector or a row vector of the sum of each column

2.4 Numeric Arrays Operations

- When dealing with numeric arrays, the **clear** function can come in handy
 - If you have **foo** declared as a 2x2 array
 - Then create two 5x1 arrays, **x** and **y**
 - Try to do: **foo(:,1) = x**
 - Produces an error because MATLAB thinks **foo** should be a 2x2 array and **x** won't fit into the first column of **foo**
 - Use **clear** to reset **foo**

2.4 Numeric Arrays Operations

- MATLAB can create several special matrices
 - Identity, zeros and ones
 - Useful for initializing matrices to ones, zeros or when you may need to use the identity matrix
- MATLAB also has functions to check for NaN, inf, or if an array is empty
 - **isnan, isempty, isinf**

2.5 Strings

- Strings are just character arrays
 - Can be multidimensional
 - However, then each string is required to be the same length to keep array rectangular

2.5 Strings

- Examples

- We've already seen strings in previous lectures, but here

```
phrase =  
a bird in hand is worth two in the bush  
  
>> phrase(1:6)  
  
ans =  
  
a bird
```

```
>>  
>> string_array = zeros(5,5);  
>> string_array(:,1) = 'abcde';  
>> string_array(:,2) = 'buddy';  
>> string_array(:,3) = 'do';  
??? Subscripted assignment dimension mismatch.  
  
>> string_array(:,3) = 'doooooo';  
??? Subscripted assignment dimension mismatch.
```

2.5 Strings

- Useful functions
 - String manipulation: **strcat**, **upper**, **lower**
 - String comparisons
 - Should use **strcmp** when comparing strings
 - If you have two strings **A** and **B**, **A==B** will give a row vector of results for each character. Here, the strings must be equal in length
 - **Strcmp** will compare the entire string, or a subset if you want

M-Files

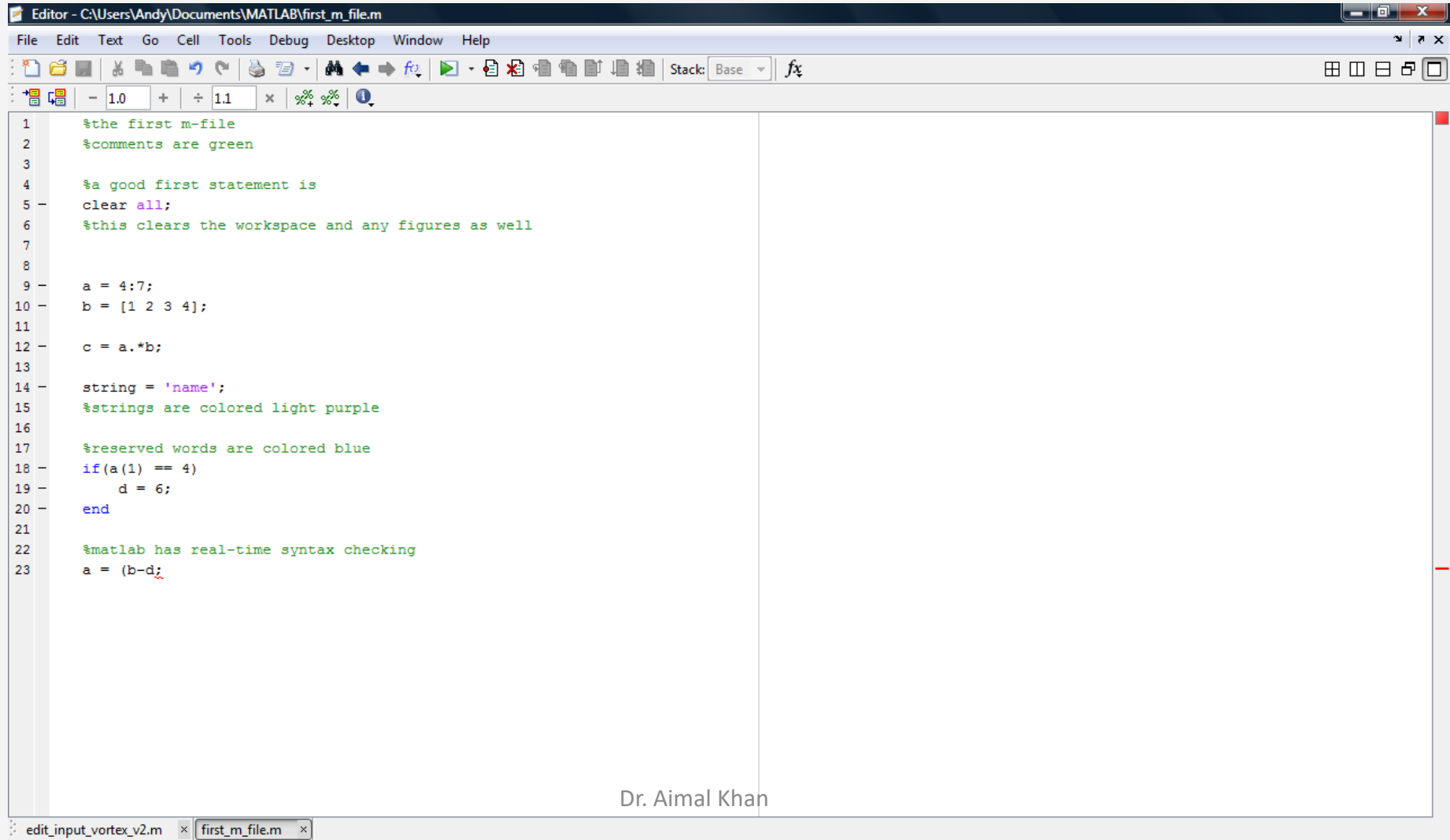
- M-files are essentially script files where you can place a bunch of MATLAB commands and execute the m-file repeatedly
 - Can have everything in an m-file, function calls, variable allocation, function definitions, figure generation and modification, etc
- Nearly everything will probably be done in an m-file, rather than at the command prompt

M-Files

- MATLAB has the editor window for creating, editing, debugging, running and saving m-files
- MATLAB color codes m-files for easier reading
- There is also real-time syntax checking
 - Essentially spell check for code

M-files

- Example m-file



The image shows a screenshot of the MATLAB Editor window. The title bar reads "Editor - C:\Users\Andy\Documents\MATLAB\first_m_file.m". The menu bar includes File, Edit, Text, Go, Cell, Tools, Debug, Desktop, Window, and Help. The toolbar contains various icons for file operations, editing, and execution. The main editing area displays the following code with syntax highlighting: line numbers 1-23 on the left, green comments starting with % on lines 1, 2, 4, 6, 15, 17, and 22, a purple string on line 14, blue reserved words on lines 5, 12, 18, 20, and 23, and black code on lines 9, 10, 11, 12, 13, 19, and 23. The code includes comments, a clear command, variable assignments, an if statement, and a final assignment. The status bar at the bottom shows "Stack: Base" and a cursor icon. The taskbar at the very bottom shows two open windows: "edit_input_vortex_v2.m" and "first_m_file.m".

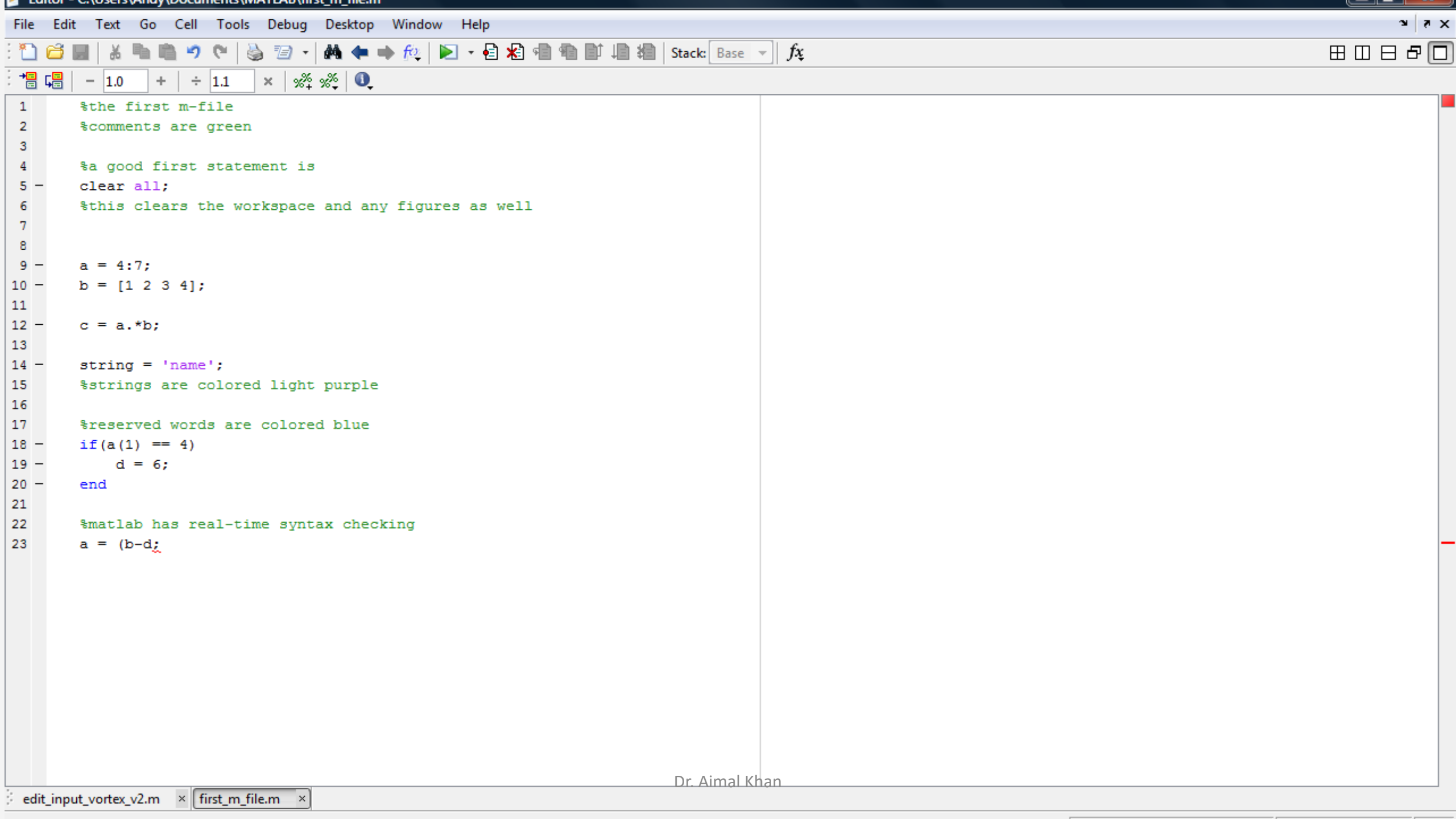
```
1 %the first m-file
2 %comments are green
3
4 %a good first statement is
5 clear all;
6 %this clears the workspace and any figures as well
7
8
9 a = 4:7;
10 b = [1 2 3 4];
11
12 c = a.*b;
13
14 string = 'name';
15 %strings are colored light purple
16
17 %reserved words are colored blue
18 if(a(1) == 4)
19     d = 6;
20 end
21
22 %matlab has real-time syntax checking
23 a = (b-d);
```

M-Files

- M-files are essentially script files where you can place a bunch of MATLAB commands and execute the m-file repeatedly
 - Can have everything in an m-file, function calls, variable allocation, function definitions, figure generation and modification, etc
- Nearly everything will probably be done in an m-file, rather than at the command prompt

M-Files

- MATLAB has the editor window for creating, editing, debugging, running and saving m-files
- MATLAB color codes m-files for easier reading
- There is also real-time syntax checking
 - Essentially spell check for code



Chapter 3 Functions

Chapter 3: Functions

In mathematics, a function is a rule that assigns to each value of the input, a corresponding output value.

Consider the function f defined by the rule

$$f(x) = x^2 \text{ for all numbers } x.$$

Here, x is the input value, and $f(x)$ is the output value.

Equivalently, we could have written the rule as

$$f(y) = y^2 \text{ for all numbers } y.$$

Functions can have many inputs and produce (through multiple rules) many outputs, $f_1(a,b,c) = 2a+3b$,

$$f_2(a,b,c) = bc.$$

FUNCTIONS in programming

If you have a “complex” task (eg., more than two lines of Matlab program code) that you will reuse in a few (or several places), you will want to write that task as a reusable function file.

Example: trigonometric SIN function

Then, every time you need to execute that task, you will only need to “call” the function.

This modularity helps break down a huge program task into a collection of smaller tasks, which individually are easier to design, write, debug and maintain.

functions are sometimes also called *subroutines*, *methods*, etc.

Types of Functions

- User Defined Functions

Functions written by you as user

- Built-In Functions

Functions which are provided by Matlab

e.g. min(), max(), sin() etc

Picture of **Addition** function



A User Defined MATLAB function

The first line is the *function declaration line*.

function [result] = addition(v,w)

The output variables. This function has One. The function's purpose is to compute these variables, based on the values of the input variables.

The input and output variables are also called the input and output arguments.

The input variables. This function has two. Within the function, the names of the input variables are **v** and **w**.

The function name, this function should be saved in a file called

addition.m

Example

Function declaration line



```
function [result] = addition(v,w)
```



```
result =v+w;
```



Logically correct expressions and assignments that compute the output variables using the values of the input variables.

Comments and blank lines add readability

```
function [sum] = addition(v,w)
% addition computes sum of two numbers

result= v+w;
```


Calling a function



```
>>
```

```
>> v = 3;
```

```
>> w = 5;
```

```
>> [result] = addition(v,w) ;
```

```
>>
```

Example: Please store the function in a .m file with name mymax

```
function max = mymax(n1, n2, n3, n4, n5)

%This function calculates the maximum of the
% five numbers given as input
max = n1;
if(n2 > max)
    max = n2;
end
if(n3 > max)
    max = n3;
end
if(n4 > max)
    max = n4;
end
if(n5 > max)
    max = n5;
end
```

Calling the function

```
Y = mymax(34, 78, 89, 23,11)
```

Built-in Functions

MATLAB has many functions called **built-in** functions.

- sin, cos, tan, exp
- reshape, size, plot

Individuals (like yourself) expand the functionality of MATLAB by writing their own m-file functions, which use the built-in functions, as well as already written m-file functions.

We will write m-file functions, using MATLAB expressions and assignments.

TYPE shows m-files

The command

```
>> type filename
```

will either

- inform you that the function is a built-in, or
- print out (to the command window) all of the lines in the m-file with that name.

Try

```
>> type sin
```

```
>> type addition
```

Builtin Functions

- Exponential
 - `exp(x)`
 - `sqrt(x)`
- Logarithmic
 - `log(x)` natural logarithm $\ln$
 - `log10(x)`
 - `log2(x)`

Builtin continued

- numeric
 - `ceil(x)` round to nearest integer towards +∞
 - `floor(x)` round to nearest integer towards -∞
 - `round(x)` round to nearest integer
 - `sign(x)` +1, 0 or -1
 - `rem(x,y)` Finds remainder of x/y
- complex
 - `abs(x)` absolute value
 - `angle(x)` in complex plane
 - `conj(x)`
 - `imag(x)`
 - `real(x)`

Builtin continued

- Trigonometric and their inverse

- $\cos(x)$ $\arccos(x)$
- $\sin(x)$ $\arcsin(x)$
- $\tan(x)$ $\arctan(x)$
- $\cot(x)$ $\operatorname{arccot}(x)$
- $\csc(x)$ $\operatorname{arccsc}(x)$
- $\sec(x)$ $\operatorname{arcsec}(x)$
-

Builtin Functions continued

- Hyperbolic and their inverse

- | | |
|----------------------------|---------------------------|
| • $\cosh(x)$ | $\operatorname{acosh}(x)$ |
| • $\coth(x)$ | $\operatorname{acoth}(x)$ |
| • $\operatorname{csch}(x)$ | $\operatorname{acsch}(x)$ |
| • $\operatorname{sech}(x)$ | $\operatorname{asech}(x)$ |
| • $\sinh(x)$ | $\operatorname{asinh}(x)$ |
| • $\tanh(x)$ | $\operatorname{atanh}(x)$ |

Builtin Functions for vectors

- `max(x)`
 - returns largest value in vector `x`
- `[a,b] = max(x)`
 - returns largest value in `a` and index where found in `b`
- same type of functions are available for min

Builtin functions for vectors

- `sort(x)` % in ascending order by default
- `mean(x)`
- `sum(x)`
- `prod(x)`
- `cumsum(x)`
 - returns vector of same size with cumulative sum of x i.e. `x=[4,2,3]` returns `[4,6,9]`
- `cumprod(x)`
 - returns cumulative products in vector of same size

Program Flow Control

- `break` Terminates execution of a loop.
- `case` Provides alternate execution paths within switch structure.
- `else` Delineates alternate block of statements.
- `elseif` Conditionally executes statements.
- `end` Terminates for, while, and if statements.
- `error` Display error messages.
- `for` Repeats statements a specific number of times
- `if` Executes statements conditionally.
- `otherwise` Default part of switch statement.
- `return` Return to the invoking function.
- `switch` Directs program execution by comparing point with case expressions.
- `warning` Display a warning message.
- `while` Repeats statements an indefinite number of times.

For Loop

```
for index = values  
    statements  
end
```

```
for index = 1:200  
    index  
end
```

Nested Loop

```
s = 10;  
H = zeros(s);  
  
for c = 1:s  
    for r = 1:s  
        H(r,c) = 1/(r+c-1);  
    end  
end
```

Loop with fractional Step

```
for v = 1.0:-0.2:0.0  
    disp(v)  
end
```

Switch Example

```
n = input('Enter a number: ');

switch n
    case -1
        disp('negative one')
    case 0
        disp('zero')
    case 1
        disp('positive one')
    otherwise
        disp('other value')
end
```


Switch Example with Strings

```
x = [12 64 24];  
plottype = 'pie3';  
  
switch plottype  
    case 'bar'  
        bar(x)  
        title('Bar Graph')  
    case {'pie', 'pie3'}  
        pie3(x)  
        title('Pie Chart')  
    otherwise  
        warning('Unexpected plot type. No plot created.')  
end
```

if else usage in matlab: conditional execution

```
a= input('Enter a Number: ')
```

```
if a < 30  
    disp('small')  
elseif a < 80  
    disp('medium')  
else  
    disp('large')  
end
```

if else usage in matlab: conditional execution

```
a= input('Enter the first Number: ')
```

```
a= input('Enter the second Number: ')
```

```
if (a < 50 && b< 50)
```

```
    disp(' a and b are small')
```

```
elseif (a < 50 && b>50)
```

```
    disp(a is small and b is large')
```

```
elseif (a > 50 && b>50)
```

```
    disp( 'a and b are large')
```

```
else
```

```
    disp('a is large and b is small')
```

```
end
```

Matlab Built-in Functions and Commands



MatlabCommands.pdf

INTRODUCTION TO BASIC PLOTTING IN MATLAB

- MATLAB has an excellent set of graphic tools.
- MATLAB provides very good tools for two-dimensional (2-D) and three-dimensional (3-D) plots.
- Plotting a given data set or the results of computation is possible with very few commands.
- Trying to understand mathematical equations with graphics is an enjoyable and very efficient way of learning mathematics.

